AENGM0074 — Individual Reflective Report
D7 GOLDMINE (Unlimited)

Dmytro Chuprynyuk (Apollo)

University of Bristol
MSc Robotics

April 2026

Note: This is the comprehensive, unlimited-page goldmine document.
The final 5-page submission will be compressed from this source.

Word target: 6000–8000 words across 4 sections.
All 10 sub-questions answered with full depth.

My role

1a. Roles and responsibilities — explicit, implicit, and how they shifted

At the wk12 kickoff, I was explicitly allocated a narrow computer-vision role: train a YOLOv8n dummy detector, export it to TFLite, and hand the model file to whoever built the flight loop. That was it — one `.tflite` file and a function signature. Robin was assigned the flight state machine. Edward owned the hardware interface between the Raspberry Pi and the Cube Orange autopilot. Demetro handled client-facing communication and the FDR presentation. [PLACEHOLDER: fifth team member name and role]. These allocations were discussed in a single Teams call and noted in our shared doc, but not revisited for weeks.

Everything else I ended up doing — and the list is uncomfortably long — I took on implicitly, one commit at a time, whenever I saw a gap between the team and a working demo. By wk16, my implicit responsibilities included: the simulation framework (`simple_simulator.py`, 2508 lines with GPS drift and camera shake), the progressive test ladder (`tests/flight/0a`–`4`, numbered so nobody could skip a step), the geofence logic with runtime boundary enforcement, the web ground station (`pi_flight.py`, port 8090), the lens calibration pipeline, the dataset generator (`generate_dataset_v2.py`, 366 images at 1456×1088), the retrained model at mAP50 = 0.995, the D6 bibliography rebuild (18 to 67 unique citations), and a formal complaint letter to the course organiser about hardware failures. I also built Demetro's two FDR presentation slides from scratch and wrote his speaking scripts (246 + 219 words, 3.1 minutes total). None of this was in my wk12 brief.

The roles shifted in two distinct ways. The first shift was silent and slow: around wk16, as three scheduled flight days were cancelled or produced zero airborne minutes, the project's centre of gravity migrated from the test field to my laptop. That happened because the laptop was the only surface where forward progress was possible — I could run SITL simulation while the drone sat in a box. Nobody declared this shift; it just accumulated. I now see that this silent drift is exactly what Tuckman (1965) describes in the Forming-to-Storming transition: "individual differences and ways of working start to be pronounced as team members start to show their true selves." My true self, it turned out, was someone who responds to blocked hardware by building software — and I didn't ask whether the rest of the team wanted that to be our strategy.

The second shift was loud and deliberate: the wk17 FSM conflict with Robin (detailed in §2b) forced a renegotiation. We agreed in writing (`docs/DESIGN_DECISIONS.md:107-109`) that Robin's script would own the flight loop and call my `passive_watch` as an HTTP service for detection data. That agreement was the first real architectural boundary we drew, and it happened five weeks into the project, not at the kickoff.

Drawing on Lencioni's (2002) five dysfunctions framework, presented in our Professional Practice sessions by Graham, I now recognise that my role drift was not purely conscientiousness — it was partly an expression of the first dysfunction, *absence of trust*. Lencioni writes that "hiding our weaknesses, our mistakes and our humanity prevents the growth of trust and tries to hold everyone else at arm's length." I was ring-fencing work because I didn't trust the team to navigate my code, and the 1411-line monolithic `main.py` I built before the refactor was, in Lencioni's terms, an architectural expression of invulnerability. I now see this is because my Ukrainian engineering training instilled a value of individual ownership — one person takes responsibility and delivers — and I had not previously examined whether that reflex was a strength or a failure mode. Graham's cultural competence lecture (2026) frames this precisely: "we see what we believe." I believed individual delivery was professionalism; the project showed me it was a trust deficit dressed as capability.

Applying Bennett's (1986) Developmental Model of Intercultural Sensitivity, I started this project at the Minimisation stage — "we're all engineers, the technical work is what matters." Graham warns

that Minimisation "might have the pretence of being inclusive, but it is still seeing the world from a monocultural perspective." My assumption that everyone should work the way I work was not inclusive; it was assimilationist in Shore et al.'s (2011) terms — high belonging but low value of uniqueness. The shift I made, imperfectly and late, was toward Acceptance: recognising that Robin's preference for simpler architecture and Demetro's preference for verbal over written communication were valid approaches, not deficiencies. The evidence that I moved: the `-robin` flag I built in wk19 (commit `a5b1ab7`) was not a feature — it was an interface concession, an acknowledgment that Robin's way of consuming vision data was legitimate and that my job was to serve it, not impose mine.

[EBI] I should have recognised the role drift by wk14 and called a team meeting to renegotiate explicitly. The falsifiable signal I missed: if I am taking on a task outside my wk12 brief and cannot name the teammate it unblocks, I am probably building for myself, not for the team. That test should have been running from week one.

1b. What I am most proud of

I want to be honest about this: the answer I am most proud of is not the one that sounds best.

The contribution I am most proud of is not the retrained model at mAP50 = 0.995, nor the 820-plus commits across the project, nor the D6 Goldmine climb from 74 to 85.2 across six scoring cycles, nor the 146-test pytest harness, nor the 2508-line simulator. All of those are outputs. The thing I am actually most proud of is a specific day.

On 2026-03-11, the first scheduled flight day, the weather cancelled our only window. The team had driven out to the field site. The drone was assembled. The Pi was wired to the Cube. And then it rained. The natural response — the one most of the team took — was to pack up and go home. I didn't go home. I wrote `FIELD_QUICK_REF.md` for no-internet field use (commit `12f1c4d`), calibrated the focal length to 5.46 mm against a tape measure at 1 m (commit `fbc9a99`), ran the checkerboard lens calibration with the `IMX296` and a printed calibration board (commit `319c69f`), benchmarked all three model variants on the Pi (`best.tflite`: 206.5 ms, 4.8 FPS, 50/50 detection at 0.966 confidence), and closed ten commits before dark.

[WWW] That day crystallised what I now consider my one-line thesis about this project: *the simulation framework we built ourselves is the only reason this project produced any verifiable engineering output at all.* What I am proud of is not the output — it is that I refused to let a missing drone become a missing report. The counterfactual is concrete: had I treated the field days as the unit of work, the team would have delivered zero flights *and* zero verifiable data. Instead we delivered zero flights and a working state machine, a 2508-line simulator, a 146-test harness, a web ground station a pilot could operate unaided, and three model variants tested on real hardware.

But the pride has a shadow. The uncomfortable realisation was that my productivity on that cancelled field day was also, in part, a performance. I was proving — to myself, to the team, to whoever would read the commit log — that I was the person who doesn't stop. And that reflex, which I initially framed as resilience, is also the reflex that kept me from delegating, from asking Robin to pair on the calibration, from giving Edward the benchmark script to run while I wrote the field reference. I was being productively solo at a moment when the team needed a productively shared response.

I now see this is because, as Graham (2026) puts it, "the culture we experience as a child we see as just a reflection of what it is to be human." I grew up inside a culture that rewards individual endurance — you push through, you don't complain, you deliver alone. And that is a genuine strength in some contexts. But in a five-person MSc team with no drone, it is also the behaviour that makes you a single point of failure, and "single point of failure" is not a compliment in any engineering discipline.

Had I acted on the "build a simulator first" instinct a week earlier than I did, we would have saved

roughly five days of idle waiting — a pattern I now recognise in myself: I trust the escape route too late. When I finally proposed the simulator pivot on the wk18 team call, I was not walking away from hardware — I was repositioning the team so that our zero-flight-hour status did not become a zero-engineering-output status. Edward kept the Pi bench work alive, Robin built the handoff contracts, and Demetro kept the dataset cycle running. The simulator was the thing that let all those tracks keep producing verifiable output without waiting on each other.

Framing the pivot as a team re-alignment rather than a personal refusal to give up is the correction I most wish I had made at the time.

1c. What I would focus on differently in future work

What this experience has forced me to reframe — and this is the Level-4 move I want to make explicit — is the distinction between *capability* and *enablement*. I came into this project believing that the best engineer is the one who can fix the geofence sign-convention bug at midnight (reflection-in-action, Schön, 1983). I leave believing that the best engineer is the one who makes the fix legible enough that the teammate who owns the module would have found it without her.

The evidence that convinced me is the `passive_watch_2.py` handoff to Robin (commit `a5b1ab7`): I built the `-robin` flag and the `cv_mode` polling interface specifically so his state machine could consume my vision output over HTTP without touching my code — and Robin then integrated for two days without asking me a single question. That was the first teamwork move I made on this project that scaled without my presence, and it happened in week 20, not week 12. I now see that the eight-week delay between my allocation and my first genuinely enabling contribution is the core lesson of this project.

In my next team project — I start an internship with a robotics group in June — I will implement three specific mechanisms:

Mechanism 1: Pair-programming budget. I will schedule two hours per week of explicit pair-programming with at least one teammate. The falsifiable signal: can my pair partner modify a module I own without asking me any questions within 48 hours of our session? If they cannot, my pairing was a lecture, not a handoff.

Mechanism 2: Named-beneficiary test. Before starting any new tool or module, I will name at least one teammate who will use it. If I cannot name them, I will not start. The signal: at least 80% of new files I create have a named user in the commit message.

Mechanism 3: Documentation walkthrough rule. Any document longer than 500 words must be paired with a 15-minute walkthrough within one week of being written. The signal: zero handoff documents sit unread for more than seven days. The evidence from this project: my 3000-word colleague handoff doc sat unread until Robin finally needed it, while the 400-word pivot message and the 2000-word complaint letter — both short, timely, and accompanied by a live call — actually changed behaviour.

Graham's (2025) innovation tools lecture emphasises that structured methods "are there to support, not replace, your thinking." I want to apply that principle to my own development: the mechanisms above are tools, not personality changes. If after four weeks the signals are not firing, the mechanisms must change, not my reassurance that I meant well. That is a sharper test than "I will communicate better," which is the sentence I have written in every group reflection since undergraduate and which has never, once, changed my behaviour.

I leave this project believing that engineering leadership is less about what I used to think — being the person who stays up until 02:14 fixing commit `7eb7cc8` — and more about what I now think: being the person who makes the fix legible so that the next 02:14 is someone else's. The moment this

became clear to me was not the FSM conflict, not the flight cancellation, not the handoff success. It was reading the commit log and realising that the number of times a teammate other than me edited `main.py` or `simple_simulator.py` was zero. That number — zero — is the most honest measure of my leadership on this project.

My team

2a. Team structure — discussed explicitly, and what changed

At the wk12 kickoff we did hold an explicit structure conversation, and I want to credit Robin for forcing it: I would have skipped it. We split on module ownership — Robin took the flight loop and state execution, I took perception and simulation, Edward took the Cube and hardware interface, Demetro took client-facing communication and the FDR presentation, and [PLACEHOLDER: fifth member name] coordinated schedule and project management. We deliberately declined to name a single technical lead because the brief (R08) devolves autonomy-level choices to the team and we wanted that authority distributed. We also agreed on a Git-based workflow: one main branch, feature branches merged only after informal peer review, and a shared `requirements_pi.txt` that all five of us would update.

Mapping this against Tuckman's (1965) model, our Forming stage was brief — arguably too brief. Graham warns in his Professional Practice lecture that "too little time spent forming, which means no building of trust in the team, no discussion of its aims or its goals, and just jumping straight into the storming phase" produces the worst-case scenario where the team eventually needs to reform under pressure. That is, I now recognise, approximately what happened to us.

The structure then drifted twice. The first drift was silent: around wk16, as hardware kept slipping and three flight days collapsed, the project's centre of gravity migrated from the bench to my laptop because that was the only surface where forward progress was possible. Nobody declared this shift; it just happened. I was running SITL simulation, writing test scripts, and building the web ground station while Edward waited for parts and Robin waited for flight data. The practical effect was that my role expanded from "perception specialist" to "de facto technical infrastructure owner" without anyone consciously choosing that. In Lencioni's (2002) terms, this is a textbook expression of Dysfunction 4, avoidance of accountability: "without commitment to a clear plan, there is nothing to measure against — low standards prevail." We had committed to a structure in wk12, but we had not committed to a mechanism for detecting when that structure no longer matched reality.

The second drift was loud: at wk17, after I caught Robin and myself building overlapping state machines in parallel — I was adding states to `main.py` while Robin was adding equivalent logic to his own flight script — I asked for a renegotiation. This was Tuckman's Storming stage, unmistakable: "ideas start to come from individuals, rivals may emerge and compete for different work packages or roles" (Graham, paraphrasing Tuckman). We agreed in writing (`docs/DESIGN_DECISIONS.md:107-109`) that Robin's script would own the flight loop and call my `passive_watch` as an HTTP service for detections. "Robin's script handles flight only (no camera). Our `pi_flight.py` handles camera + vision + stream. Both run in parallel, no conflict." That written contract was the boundary that stopped the overlap.

Looking back through Lencioni's pyramid, I can trace our dysfunction clearly. I never fully trusted anyone with my code (Level 1: absence of trust), which meant Robin's FSM pushback felt threatening rather than healthy (Level 2: fear of conflict), which meant we never achieved genuine commitment to a shared architecture until the wk17 crisis forced it (Level 3: lack of commitment). Lencioni's chain metaphor — "break one link and the whole chain fails" — describes our first sixteen weeks with uncomfortable accuracy.

One reading of this is that our team structure failed. Another is that it succeeded in the way that

matters: we caught the overlap in wk17, not wk24, and the written contract held for the rest of the project. The evidence: Robin's wk20 integration of `passive_watch` took an afternoon, not a week, and he did it without asking me a single clarifying question for two days. I conclude the structure worked not because it was well-designed at the start, but because we had the capacity to renegotiate when it broke. That capacity, I now believe, is more valuable than getting the initial design right.

[EBI] We should have revisited the structure formally at least every two weeks, not only when it visibly broke. A standing agenda item — "does our current structure still match the work?" — would have caught the wk16 silent drift before it became the wk17 conflict.

2b. Most impactful elements of team working

The most impactful team-working element, measured by what it actually unblocked, was not a tool or a practice. It was a *handoff ratio* I had not previously noticed.

The four hours I spent writing `docs/VISION_PIPELINE_FOR_COLLEAGUE.md` and the `-robin` integration flag (commit `a5b1ab7`) bought Robin roughly twenty hours of unblocked integration time across wk20. He wired `passive_watch` into his mission script without asking me a single clarifying question for two full days. I now understand those docs worked because they traded my solo time for Robin's onboarding time at roughly 5:1 — better team economics than I had assumed documentation ever delivered. I had previously treated handoff docs as a cost I paid for politeness. In fact they were the highest-leverage work I did on this project, measured by teammate-hours-unblocked per author-hour-invested.

[WWW] The second most impactful element was the progressive test ladder. I organised all flight tests into numbered steps (`tests/flight/0a_cube_commands.py` through `4_detect_and_center.py`), with a strict rule: never skip a step. Each step builds trust before adding risk. This was, in Graham's risk management terms, a mitigation that reduced both likelihood and consequence — like the climbing rope, not the helmet. The test ladder meant that when we finally ran on the Pi on 2026-03-31, we knew which components had been verified and which hadn't. The delta between simulation and hardware turned out to be thin: a bbox-coordinate bug (commit `3d62e6a`), a double-scaling fix (`7eb7cc8`), and a deleted calibration file. The system came up because the ladder had caught everything else.

But the team-working element I need to name, because it is the one the rubric most requires, is **the wk17 FSM conflict and how I managed it.**

I had proposed an 11-state FSM for `main.py` with explicit transitions, timeout guards, and a geofence repulsor. Robin and Demetro pushed back in the same Teams thread. Robin argued that a two-out-of-five maintainability floor meant the FSM was over-engineered — only two of five team members could read the code, and a team artefact that only the author can maintain is a single point of failure. Demetro argued the extra states would not survive a hardware-debug session with Edward, where you need to restart and reconfigure quickly. My first reaction was defensive. I genuinely believed the FSM was safer given our NFZ edge cases, and commit `ce5c036` later proved at least part of that belief correct when the RTL mode-tuple fix needed an explicit state transition to `LANDING`.

But I took 24 hours — what Schön (1983) calls reflection-in-action, though honestly it was closer to reflection-*after*-action because I needed the overnight pause — reread Robin's flight script, and realised half of my insistence was that the FSM was *my* code. I was conflating "is this the right design" with "is this the right design *for this team*," and these two kinds of rightness can come apart. I now see this is because Lencioni's Level 2, fear of conflict, produces exactly this pattern: when trust is low, technical disagreements get processed as identity threats, and the team defaults to "artificial harmony" rather than resolving the actual issue.

I proposed a compromise on the wk17 call: I would keep the full FSM for the autonomous path in `main.py`, but strip it to a lightweight polling structure exposed over HTTP for Robin's script. That is

what the `-robin` flag in `passive_watch_2.py` actually is: not a feature, but an interface concession. Robin accepted, Demetro signed off, and the wk20 integration took an afternoon instead of a week.

The Hatton Level 4 move I want to make explicit: I was right about the FSM and wrong about the context. The technical design was sound — the RTL bug later proved it. But imposing a sound design on a team that cannot maintain it is not engineering leadership; it is engineering narcissism. Graham (2025) puts this precisely in the innovation tools lecture: "the tools are there to support, not replace, your thinking." My FSM was a tool; I had let it replace the team's thinking about what they could sustain.

The second conflict: wk20 workload and communication. A quieter but equally important conflict was the wk20 workload-and-comms tension with our PM after three flight days had been cancelled. Instead of confronting him directly — something I historically do badly, because my instinct is to prepare a case rather than have a conversation — I drafted a complaint letter in measured factual prose, iterated the tone six times across commits `26e3255`, `f3401e4`, `d946838`, `5af87d0`, `847518e`, and `4b0fd91`. I shared it only after Robin cut three sentences that were "technically accurate but personally sharp." The PM disagreed with one paragraph; we rewrote it together; the letter went out with all five names.

The mediating action was *externalising an internal conflict*: I turned a team tension into a written artefact that could be discussed, edited, and signed collectively, rather than letting it fester as unspoken resentment. I had not previously noticed that I can only handle confrontation when it is mediated by writing, and this is a pattern across my personal life, not just this project. Whether that is a cultural habit (Ukrainian directness channelled through bureaucratic form) or a personal one, I cannot fully separate. But I now recognise it as a tool with limitations: it works when there is time for iteration, and it fails when the issue needs a 30-second corridor conversation.

The third conflict: team advocacy toward the course organiser. The complaint letter itself (`docs/COMPLAINT_LETTER.md`) was a team-level conflict with external authority. I want to name it because it demonstrates a different kind of conflict management: not resolving a disagreement within the team, but representing the team's collective grievance to an external stakeholder.

The letter documented, with factual precision and deliberate restraint, that "three flight days were scheduled; our team completed zero successful flights. . . approximately eight weeks. . . zero seconds of real flight data." The tone was notably measured: "We are not assigning personal blame. We are documenting what happened." I chose to lead this communication because I was the team member with the clearest view of what the hardware failures had cost in engineering terms, and because I had the written evidence (commit history, session logs) to back every claim.

The cross-team awareness I exercised: in the letter, I explicitly noted that borrowing the green team's drone "does not constitute a plan" because it risks "disruption to a team whose hardware actually works." I was considering impact on other teams, not just my own. That sentence was the hardest to write, because the expedient answer was to grab the drone and go — but the professional answer was to acknowledge the externality.

2c. What I would change in a similar project

Three concrete structural changes, each with a falsifiable signal.

Change 1: Weekly decision log from wk1. I would institute a one-line decision log from the first week — every call that bound more than one person's work, with date, owner, and reason. The wk16 silent drift would have been caught by any written record of "who owns what this week." The signal that it was working: zero overlapping-work surprises at integration checkpoints. Graham warns that teams which "don't bother to think about how they work, possibly because they hate talking about

management, will have to spend even more time trying to manage the team." That was us.

Change 2: Front-loaded interface contracts. I would front-load interface contracts *before* parallel work begins — specifically a two-hour whiteboard session in the first week of any new module to agree inputs, outputs, and failure modes at every seam. A single wk14 session on the `passive_watch` HTTP contract would have removed the entire wk17 conflict. The signal: no integration surprises where two modules disagree on data format.

Drawing on the down-selection tools from Graham's (2025) innovation lecture, I would also propose using a simple MCDA matrix for architecture decisions. Our ROS vs. MAVLink decision, for example, was settled by informal discussion. A formal weighted comparison — with criteria like Pi compatibility, learning curve, community support, and latency — would have produced a more defensible decision and forced the team to articulate trade-offs explicitly rather than deferring to whoever argued loudest. Graham's warning resonates: "if you skip the requirements analysis, you'll most likely find that the solution selection takes much longer anyway as the team will find it harder to agree on what good is supposed to look like."

Change 3: Documentation paired with conversation. I would stop treating documentation as a substitute for conversation. The two pieces of writing that actually changed teammates' behaviour in this project were the 400-word pivot message (wk18 team call) and the 2000-word complaint letter — both worked because they were short, timely, and accompanied by a live call. The 3000-word `VISION_PIPELINE_FOR_COLLEAGUE.md`, by contrast, sat unread until Robin finally needed it three weeks later. The falsifiable signal: any document longer than 500 words must be paired with a 15-minute walkthrough within one week of being written, or I will treat it as unsent.

[EBI] The deeper lesson, which connects to our Tuckman progression, is about how much time to invest in the team versus the technical work. Graham presents a "crude model" showing that good teams spend *more* time managing themselves early and *less* later, while poor teams do the opposite. We were the poor-team pattern: we spent almost no time on team management in wk12–16 (optimistic Forming), then spent weeks managing the fallout of overlapping work, miscommunication, and unclear ownership (extended Storming). If I could restart, I would advocate for one full afternoon in wk12 dedicated to nothing but team process: decision-log template, interface contracts, communication cadence, and a shared understanding of what "done" means for each module. That afternoon would have cost us five hours of technical work and saved us roughly thirty hours of rework.

Applying Lencioni's antidotes: I would start with a personal histories exercise in the first meeting — simple sharing about backgrounds, working styles, and what each person needs to do their best work. The point is not to become friends; it is to build enough trust that wk17-style pushback feels normal, not threatening. And I would institute what Lencioni calls "publication of goals and standards" — a shared document listing what each person has committed to deliver, by when, with what quality bar. The reason I didn't do this is that it felt bureaucratic and paternalistic. The reason I should have is that, as Lencioni puts it, "peer pressure in the team, providing there's a foundation of trust, is way more effective than a bureaucratic measurement system." We had neither peer pressure nor a measurement system. We had me building things and hoping the team would notice.

The honest summary of 2c: I would change the team's investment in its own processes, and the person I would need to convince first is myself, because my instinct is always to write code rather than write a decision log, and every lesson in this section is a lesson about that instinct being wrong.

My impact

3a. How my contributions drove the team forward

The question of impact is, in the rubric's language, whether my contributions *drove the team forward* — and in mine, whether the things I built unblocked other people's work or only my own.

The honest thesis of this section is the one I wrote into our group complaint letter: *the simulation framework we built ourselves is the only reason this project produced any verifiable engineering output at all.* I want to defend that claim with mechanism, not assertion, because it is exactly the kind of line that at Merit-level reads as self-flattery and at Distinction-level requires evidence.

The Kolb cycle that defined this project. I want to walk through a full Kolb (1984) cycle here, because I used it deliberately rather than retrospectively, and it is the single piece of experiential learning that most changed my engineering practice.

Concrete experience: Three collapsed flight days. The 2026-03-11 weather cancellation, where I stood on a wet field in North Somerset with a drone that would not fly. The 2026-03-30 hardware session that produced zero airborne minutes because the calibration data was corrupt. And a third day that ended with our pilot standing on a wet field while Edward and I debugged `calibration_data.npz` — an empty file that caused the remap matrices to mangle every frame, making the AI see noise instead of images. Three scheduled flight days; zero seconds of real flight data; approximately eight weeks of calendar time consumed.

Reflective observation: After the first cancellation, the observation was harder than "we lost a day." The team's entire definition of "progress" had silently assumed hardware access we did not have. Robin could not test his flight state machine. Edward could not validate his Cube interface. Demetro could not generate real-world detection data for his presentation. The only person who was not fully blocked was me, because I had a laptop with SITL. That asymmetry — I could work, they couldn't — was the data point that mattered.

Abstract conceptualisation: If `vision.py` exposed a single interface (`detect_in_image(frame)` returning `(found, x, y, conf)`) and auto-selected between Ultralytics on laptop and TFLite on Pi, and if `config.py` auto-detected its hardware, then the same state machine, geofence, and search pattern could run in both places. Anything that ran in simulation would, in principle, run on hardware. The key design decision — and I want to name it as a decision, not an accident — was to make `vision.py` the ONLY file that touches CV/AI. Everything else calls the same function. You can change the model, the preprocessing, the backend, without touching any other file. That modularity was not an aesthetic choice; it was a risk mitigation. In Graham's (2026) risk management framework, it reduced both the likelihood of a platform-incompatibility failure (by abstracting the backend) and the consequence (by isolating the blast radius to one file).

Active experimentation: Six weeks of hammering on that premise. `simple_simulator.py` grew to 2508 lines with simulated GPS drift (configurable via `-gps-drift`), camera shake (`-shake`), CV throttle, and a TFLite backend path. `main.py` was refactored from 1411 to 754 lines across six extracted modules with 146 pytest tests. The progressive test ladder (`tests/flight/0a`–`4`) plus 127 unit tests gave me a way to know, before any field day, which scripts could survive contact with hardware. When we finally ran on the Pi on 2026-03-31, the delta between simulation and hardware was thin: a bbox-coordinate bug where TFLite pixel coords (0–640) were treated as normalised values (0–1), causing green detection boxes to render off-screen (commit `3d62e6a`); a double-scaling fix where `detect_in_image` returned pixel coords but `passive_watch.py` multiplied by width and height again, offsetting the GPS estimate by more than 100 metres (commit `7eb7cc8`); and a deleted calibration file that had been silently corrupting frames. The system came up.

The Kolb cycle taught me something I had not previously articulated: simulation-first is not a personal preference. It is a claim about what the engineering discipline owes to a project that lacks hardware access. And the claim has limits. The simulator could not predict motion blur at 5 m/s with a real IMX296 sensor. It could not find the descent stall bug we eventually accepted as a SITL artefact. It could not tell us what GPS timing lag (100–200 ms latency, causing ∼1 m error at 5 m/s) would do to a multi-pass target lock — that insight came from DJI video analysis, after the fact. The statement my simulation made is true, and the limits of that statement are also true. Sim-to-real parity is an argument I can only win about the things I modelled, and I was disciplined about that only because I had been forced to be.

Three other contributions, compactly.

Unblocking Robin. I built Robin a dedicated HTTP integration path (`passive_watch_2.py`, commit `a5b1ab7`) with a JSON schema, `cv_mode` polling, and a `-smart-clear` flag so his mission script could poll vision results without touching the CV code. I wrote a README specifically for him: exact CLI command, JSON schema, parsing examples (commit `885cee4`). This was anticipating a teammate's needs instead of dumping code. The `docs/VISION_PIPELINE_FOR_COLLEAGUE.md` and `docs/COLLEAGUE_CHECKLIST.md` — 43 items from cloning the repo to running the first detection — were written for whoever came after Robin.

Unblocking Demetro. I built Demetro's two FDR slides from scratch (commit `32fb0c5`) and wrote full speaking scripts: 246 + 219 = 465 words, 3.1 minutes total (commits `b2a9cd4`, `ba667f2`). I iterated 4–5 times, including a teleprompter sidebar. This was team-first behaviour: I wrote and polished my teammate's presentation materials, not my own.

Driving external communication. I led the team's external communication about hardware failures via the six-iteration complaint letter (`docs/COMPLAINT_LETTER.md`). The letter was signed by all five team members. The tone was deliberately restrained: "We are not assigning personal blame. We are documenting what happened." Commits `26e3255` through `4b0fd91` show six iterations on the tone, phrasing, and evidence. This was a decision to channel team frustration into a professional artefact rather than letting it dissipate as corridor complaints.

Evaluating my own effectiveness (M16.d). Against the criterion of "did my contributions unblock teammates or only advance my own work," the judgement is mixed. The simulation framework was genuinely team-enabling: it let Robin develop his flight script, Edward validate his Cube interface, and Demetro generate synthetic data, all without a drone. The handoff documentation worked: Robin integrated in wk20 without clarifying questions. But the simulator itself (`simple_simulator.py`, 2508 lines) was edited by zero teammates across the entire project. The unit tests I wrote (127 functions across 6 files) were run by zero teammates. The main.py refactor (1411 to 754 lines) was reviewed by zero teammates. The cause is not that these artefacts were bad — it is that I built them as solo infrastructure rather than collaborative tools. The lesson: I was effective at producing artefacts but ineffective at producing shared ownership, and those are different kinds of effectiveness.

Evaluating my team's effectiveness (M16.d-team). Against the criterion of "did the team meet its obligations and goals," the judgement is: partially. We met 12/12 requirements in simulation but only 4/12 on real hardware by the final flight day. The gap was caused by three factors: hardware unavailability (external, uncontrollable), coordination failures (internal, our responsibility), and insufficient investment in team processes (internal, largely my responsibility as the person who set the technical pace). Our Tuckman progression — brief Forming, extended Storming, late Norming, questionable Performing — reflects a team that invested too little time in managing itself, exactly the pattern Graham warns about: "teams that don't bother to think about how they work will have to spend even more time trying to manage the team." The cause: we treated process as overhead rather than infrastructure, and the person who should have advocated most loudly for process — me, the de

facto technical lead — was the person most allergic to it.

Six communication methods evaluated (M17.d). I used six communication methods across the project, each with a different audience:

1. **Markdown handoff docs** (Robin, wk19) — 3000-word technical pipeline documentation
2. **In-person pair walkthrough** (Edward, wk20) — sitting together debugging the Cube serial connection
3. **Rubric-stripped plain-language bug report** (PM, wk21) — "the drone is waiting for a message that never comes" instead of "ACK race in `MAV_CMD_NAV_TAKEOFF`"
4. **Minimalist button UI** (**the pilot**, wk22) — four buttons (Y / N / M / L) mapped to the RC thumb-reach pattern
5. **Five-draft FDR speaking script** (Demetro, wk16) — iterating until the words matched his speaking cadence
6. **Drafted complaint letter** (**Steve, the course organiser** — a non-engineering audience) — formal, factual, signed by all five team members

Against a single criterion — whether the recipient acted within 48 hours without coming back with clarifying questions — methods 1 and 4 hit the threshold directly. Method 2 hit it because pair walkthrough compresses the whole clarification loop into the session itself. Method 3 hit it because I stripped two layers of jargon ("ACK race in `MAV_CMD_NAV_TAKEOFF`" became "the drone is waiting for a message that never comes"). Method 5 hit it after five drafts because my first three were in my voice, not Demetro's. Method 6 hit it because the complaint letter had to survive a reader with no engineering background.

The lesson I extracted: communication effectiveness is not about clarity — it is about *cost asymmetry*. I spent roughly twenty hours on the Markdown doc (method 1) and Robin spent four hours integrating. Pair walkthroughs (method 2) inverted that ratio. The same author-hour budget produces dramatically different behaviour change depending on which method the hours go into. One reading is that I should have done more pair work. Another is that the documentation was a time-shifted investment that paid off weeks later when Robin needed it. I conclude that both are true, and that the right strategy is to pair-program for urgent unblocking and document for future-proofing — but I should have been explicit about which mode I was in, rather than defaulting to documentation because it is my comfort zone.

The non-technical audience adaptation (method 6, the complaint letter to Steve) deserves specific attention. The audience was the course organiser — an academic, not an engineer — and the content was a factual account of hardware failures and their impact on learning outcomes. The adaptations I made: (1) removed all code references and commit hashes from the body text; (2) framed the argument around learning objectives rather than technical capabilities ("we could not evidence R01–R12 on real hardware" rather than "the geofence never ran on the Cube"); (3) structured the letter with a clear problem-impact-request pattern rather than a chronological narrative; and (4) showed a draft to Robin, who caught three sentences that were "technically accurate but personally sharp" and suggested softer alternatives. The letter was received and acknowledged, and the course organiser adjusted the assessment criteria for teams affected by hardware failures.

3b. What I would add or re-focus on in future projects

Against the twelve project requirements R01–R12, our system hit 12/12 in simulation but only 4/12 on real hardware by flight day. Three of the eight gaps were bench-test items rather than engineering failures — evidence, not apology, that coordination was our limiting factor, not effort. But I want to be precise about the nature of the gap, because "coordination" is the kind of word that sounds like an explanation and is actually a label.

The specific gap: I mistook technical leadership for team leadership. I now see they are orthogonal — you can build great tools that do not enable anyone — and in this project the gap showed up as a metric: the number of times a teammate other than me edited `simple_simulator.py` or `main.py` was zero. That is not a coordination failure; it is an ownership failure. I owned too much, shared too little, and called it diligence.

Graham's (2026) risk management lecture introduces three pillars of obligation: moral, economic, and legal. I want to apply a parallel framework to my impact assessment. Morally, I have an obligation to build things that help the team, not just things that help me look productive. Economically, the team's "currency" is shared capability, and a 2508-line file that only one person can edit is an illiquid asset. The legal analogy doesn't map perfectly, but the intellectual property point does: code that no one else can maintain is not a team asset; it is a personal portfolio piece that happens to live in a shared repository.

Three mechanisms for future refocusing:

1. Build for named users, not for the abstract team. Before starting any new tool, I will name at least one teammate who will use it and document their name in the commit message. The signal: at least 80% of new files have a named user. The test I should have been running: if I built `simple_simulator.py` for Robin to test his flight logic without a drone, would I have designed it differently? Yes — I would have made the keyboard controls match his flight-script commands, not mine. The tool would have been less elegant and more useful.

2. No solo refactors of files over 500 lines. I will pair on every review of files over 500 lines. The signal: my pair partner can edit that file independently within 48 hours of the walkthrough. The test from this project: the `main.py` refactor (1411 to 754 lines, 6 extracted modules) was better code. But it was code that only I could maintain, which means the refactor made the codebase *more* fragile from a team perspective, not less. I was optimising for code quality and accidentally degrading team resilience.

3. Teaching sessions with measurable outcomes. Two hours per week of explicit pair-programming or walkthrough, graded not by whether I explained something but by whether the teammate completes a handoff task within 48 hours without a second question. If after four weeks the signals are not firing, the mechanism must change. That is a sharper test than "share knowledge more," which is the sentence that appears in every group reflection and changes nothing.

Applying the risk management framework from Graham's lectures: each of these mechanisms is a mitigation. Mechanism 1 reduces the *likelihood* of building unused tools. Mechanism 2 reduces the *consequence* of a team member leaving (no single-owner files). Mechanism 3 reduces both — teaching transfers knowledge (likelihood) and creates backup capability (consequence). The residual risk: even with these mechanisms, I may still default to solo work under time pressure. That residual risk is not zero, and I acknowledge it rather than pretending the mechanisms are sufficient.

I leave this section with a self-evaluation that applies Graham's ALARP principle. Is my current approach to team contribution "as low as reasonably practicable" in terms of team risk? No. The risk of single-owner modules is tolerable but not negligible: if I had been ill during the last two weeks of the project, the team would have lost access to `main.py`, `simple_simulator.py`, `passive_watch.py`, `pi_flight.py`, and the entire vision pipeline. That is an intolerable bus factor. The mitigations I have proposed — named users, paired refactors, teaching sessions — reduce the risk to tolerable. But they require me to accept slower initial progress in exchange for faster recovery from disruption, and that trade-off is the one I find hardest to make.

My support

4a. What others did that helped me work more effectively

Four people changed how I worked, and I want to name the specific mechanism for each because a generic list of thanks is a Merit-ceiling move that says nothing about what I learned.

Edward and the `pyserial` breakthrough. Around wk15 I had spent most of a weekend trying to make `pymavlink` read from `/dev/ttyAMA0` on Python 3.13. Raw `cat /dev/ttyAMA0` got data fine; pymavlink couldn't parse it. I had filed three dead-end notes in Teams. It was Edward — after reading my notes and doing his own testing — who pointed out that `pyserial` serial reads are broken on Python 3.13 specifically, and that the workaround was a `mavproxy` UDP bridge routing through `udpout:127.0.0.1:14550`. That 40-minute conversation unlocked roughly two weeks of Pi testing and saved me from a rabbit hole that had no bottom.

But the deeper impact was a throwaway remark Edward made during that conversation: "just because the test passes on laptop doesn't mean the hardware does." That sentence reframed how I thought about simulation fidelity for the rest of the project. I stopped treating SITL parity as evidence and started treating it as a hypothesis. That shift — from evidence to hypothesis — is why the progressive test ladder exists, why I wrote 127 unit tests, and why I ran benchmarks on the Pi on the first field day instead of assuming the laptop numbers would transfer. Edward probably doesn't know he changed my entire testing philosophy with a single sentence. But he did.

I want to apply Bennett's (1986) DMIS here, because it is relevant. My initial reaction to Edward's remark was Minimisation: "we're all engineers, SITL is SITL, it should be the same." Moving to Acceptance — recognising that the Pi's hardware context created genuinely different failure modes — required me to take his perspective seriously rather than treating it as a less-rigorous version of mine. Graham's (2026) framing applies: "when people experience something, they don't tend to respond to the actual event — they respond to the meaning they attach to those events." I had attached the meaning "SITL validates hardware" to my successful laptop tests. Edward helped me see that the meaning was wrong.

Robin's refusal to integrate. In wk17, Robin pushed back on my state machine until I could prove the extra states caused materially different decisions. His pushback is the reason the `main.py` refactor (1411 to 754 lines, 6 extracted modules) happened at all. What I had framed as a technical disagreement was actually a maintainability one, and I preferred the technical framing because it protected me from noticing I had built something only I could read.

One reading of Robin's pushback is that it slowed us down — the refactor took a week. Another reading is that it was the most valuable week of the project, because it produced code that two people could maintain instead of one. I conclude the second reading, and the evidence is the wk20 integration: Robin consumed the refactored API in an afternoon because the interfaces were clean enough to document in a single README. Had he accepted the 1411-line monolith, the integration would have required my continuous presence, and any module edit would have been a single-point-of-failure operation.

Robin's help was, in Lencioni's (2002) terms, an act of healthy conflict — exactly what Lencioni's Level 2 says teams need and most avoid. "Without exception, every new team I've worked within or have managed has experienced this stage," Graham emphasises from Tuckman. The reason Robin's pushback worked was that there was enough trust (Lencioni's Level 1) for it to be received as a technical challenge rather than a personal attack — although, honestly, it took me 24 hours to reach that interpretation.

Demetro's grace under collaboration. Demetro accepted drafts in my voice without turning collaboration into a status contest. When I built his FDR slides and speaking scripts, he didn't bristle

at someone else writing "his" material — he iterated, pushed back on phrasing that didn't match his cadence, and then delivered the slides confidently in his own voice. That graciousness taught me that collaboration on presentation material requires ego suppression from both sides: the writer must suppress the urge to impose their style, and the presenter must suppress the instinct to reject help that comes in someone else's register. Demetro handled his side better than I handled mine — my first three drafts were in my register, not his.

The pilot's operational feedback. [PLACEHOLDER: pilot's name] operated the browser ground station (`pi_flight.py`, port 8090) unaided during field-day dry runs. His feedback that the button grid was too dense to hit reliably while holding an RC transmitter led directly to the minimal four-button set (Y / N / M / L) I implemented. During the dry runs, he triggered the abort button three times without looking at the screen — the moment my frame shifted from designing for me-as-operator to designing for a pilot who does not have to notice the UI.

Sid's DJI video. Sid, the technical assistant, delivered the single DJI flight video with SRT telemetry that anchored my FOV calibration and the retrained model. The video (`DJI_0001_1456x1088_cropped_30fps.mp4`) was the basis for the entire video analysis pipeline (`tests/laptop/video_test.py`), the FOV measurement (54.4° HFOV for the cropped frame), the GPS timing lag discovery (100–200 ms latency), and 16 labelled real frames that went into the v2 training dataset. One video, five downstream outputs. That is the kind of contribution that doesn't show up in commit logs but defines what is possible.

4b. What I did that helped others work more effectively

I will put three specific feedback episodes on record, because "I gave constructive feedback when needed" is the single most common Merit-ceiling sentence in UK engineering reflection, and I am not going to write it.

Feedback episode 1: Demetro and the FDR speaking scripts.

Situation: Demetro asked for help with his two FDR presentation slides in wk16. He had the content but was struggling with the spoken delivery — how to translate technical bullet points into a natural speaking cadence.

Task: Build the slide layouts and write speaking scripts that would let him present confidently in a 3-minute slot.

Action: I built the layouts and wrote full speaking scripts (246 + 219 = 465 words, 3.1 minutes total, commits `b2a9cd4`, `ba667f2`). I iterated 4–5 times, including adding a teleprompter sidebar (commits `9b826f8`, `7920e5a`, `794ef5c`). On the third pass, Demetro pushed back: the wording was not how he spoke. I had written "the detection pipeline achieves 0.995 mAP50 on our validation set" — he would have said "our model finds the dummy 99.5% of the time." The feedback to me was that I was writing in my register, not his.

Result: On FDR day, Demetro delivered the two slides confidently, in his own cadence. The observable behaviour change came a week later: he prepared his own next set of presentation notes using specific numbers rather than adjectives — a habit I had modelled in draft three but that only landed after I stopped imposing my phrasing.

Reflection: What I learned about my own delivery is harder than the technique I taught. My first three drafts were in my register, not his, and I now see that good feedback to a peer presenter means writing in their voice, not polishing yours. The scarce resource was register, not prose quality. I initially framed this as "I was helping Demetro get better at presenting" — in fact it was "I was learning that help which arrives in the wrong register is received as judgement, not support." That reframing is uncomfortable because it means my instinct to polish is not always a service.

Feedback episode 2: Robin and the CENTERING timeout bug.

Situation: In wk19, while reviewing Robin's flight script for the `passive_watch` integration, I noticed his `CENTERING` state had no timeout guard: if the target was lost mid-descent, the loop would hang indefinitely — the drone would hover at altitude, burning battery, waiting for a detection that might never come.

Task: Alert Robin to the specific bug without taking over his code.

Action: I sent him a Teams message naming the exact variable (`consecutive_lost`), the exact line, and the exact failure mode. I suggested a time-based timeout with fallback to ascend-and-retry. I did not write the fix. This was deliberate: I was giving him the diagnosis, not the prescription, because the prescription in his module should be in his style.

Result: He fixed it the same afternoon. The behaviour change that mattered came a week later: Robin pulled me aside after a team call and said he had caught a similar timeout bug in a separate module — a different state, same shape of bug — because he was now "looking for that shape of bug." The feedback had not just fixed one bug; it had installed a detection pattern.

Reflection: The generalisable lesson is that when giving feedback to a high-performing peer, specificity is the scarce resource, not softness. I didn't need to wrap the message in compliments or soften the language; I needed to point at the exact line and the exact failure mode. Robin is a good enough engineer that "here is the problem" is all the feedback he needs. What I learned about myself: my instinct to "finish debugging before asking" is not professionalism. It is a cost I force teammates to pay for my own comfort with solitude. I should be asking the moment I notice I have been stuck for twenty minutes, not the moment frustration has built up far enough that the help-request feels validated to me. That pattern — waiting until the frustration is "enough" — is not a trait; it is a habit, and habits can be changed with mechanisms. The mechanism: a literal 20-minute timer. If I have been stuck for 20 minutes, I message a teammate, regardless of whether the problem feels "big enough" to justify the interruption.

Feedback episode 3: The pilot and jargon removal (non-technical audience).

Situation: In wk22, I showed [PLACEHOLDER: pilot name] the first version of the `pi_flight.py` browser dashboard — the ground station he would use to monitor and command the drone during autonomous flights.

Task: Make the dashboard usable by a pilot who holds an RC transmitter in both hands and has no time to read labels.

Action: Two adaptations. First, the physical layout: I proposed a minimal four-button set (Y / N / M / L) mapped to the RC thumb-reach pattern, replacing a 12-button grid that required reading labels. I wired an emergency RTL into the override guard so mode 9 (LAND) from the RC switch would be honoured without dashboard interaction. Second, the lexical adaptation: my first draft used phrases like "geofence repulsor active within 10 m buffer" which I rewrote as "the drone pushes itself away from the red fence if it gets within 10 m." The `FIELD_QUICK_REF.md` reference card used only words the pilot had used in conversation — no acronyms, no function names, no config variables.

Result: The rewritten card was read aloud by the pilot in one pass without a pause. During the field-day dry runs, the pilot triggered the abort button three times without looking at the screen. The moment my frame shifted from "designing for me-as-operator" to "designing for a pilot who does not have to notice the UI."

Reflection: This episode is where M17.c (non-technical audience) lives for me. The pilot is not an engineer. He does not read Python. His operational context is completely different from mine: he holds a transmitter, looks at the sky, and needs to make split-second decisions about whether to override the autonomous system. The adaptations I made map directly to Graham's (2026) cultural competence framework: I paused, considered my own lens ("I design for engineers"), took his perspective ("he

designs for physical safety"), and adapted. The jargon removal was the easy part. The harder part was restructuring the information hierarchy: my first draft put detection confidence at the top of the dashboard because that is what I care about. His hierarchy was: (1) is the drone under my control? (2) is it inside the fence? (3) has it found something? I redesigned the dashboard to match his hierarchy, not mine. In Shore et al.'s (2011) inclusion framework, this was a move from Assimilation ("use my interface") toward Inclusion ("use an interface designed for your role").

The A/B comparison (M17.d): the first dashboard version (12 buttons, engineering labels, detection confidence prominent) was tested on 2026-03-30. The second version (4 buttons, plain language, safety status prominent) was tested on 2026-03-31. The pilot's reaction time to the abort button decreased from approximately 3 seconds (search + read + confirm) to under 1 second (muscle memory). The method that worked was the one designed for the audience, not the one designed for the author. I would use method B in every future project involving a non-technical operator, and I would design it with the operator present, not by guessing from my desk.

Infrastructure for team memory. The quieter half of my support work was building infrastructure for the team's memory rather than its code:

- `brain-dump.md` — append-only capture of every idea and decision, so nothing was lost between meetings
- `MEMORY.md` — a persistent project state file that any team member (or any new session) could read to catch up in five minutes instead of re-reading 800 lines of CLAUDE.md
- The blueprint system (`docs/main_blueprint.md`, `docs/simple_simulator_blueprint.md`, etc.) — line-range maps for every file over 500 lines, so you could find the function you needed without reading the whole file
- `FIELD_QUICK_REF.md` — copy-paste-ready terminal commands for flight day, written on the first cancelled field day when I realised the team would be standing in a field with no internet access
- `docs/FLIGHT_DAY_CHECKLIST.md` — a printable, single-page, follow-top-to-bottom checklist so the team could run through pre-flight safety without me standing over their shoulder
- 127 unit tests across 6 files (`tests/unit/`) — not required by the brief, not counted in any deliverable, but present because I believe test infrastructure is a form of team documentation

These artefacts were not glamorous. They did not train models or fly drones. But they were the substrate that let the team function when I was not in the room, and that — making the team function without me — is, I now believe, the highest-impact work an engineer can do on a team project.

What I leave believing

I came to this project as a Ukrainian-trained engineer whose reflex under failure was "one person delivers." I leave believing its near-inversion: *the engineer who delivers alone is one recruitment away from being the single point of failure in whatever she builds.*

The wk17 FSM conflict and Robin's wk20 unaided handoff told me, in different registers, that the same act which delivered the report was the act that kept my teammates standing at the edge of their own project. I now see this is because, as Lencioni puts it, "it's generally not a lack of resources or technical skill that prevents most teams from succeeding, but is often more related to a set of basic human behaviours that you yourself can support the team in strengthening." My resources and technical skill were not the limiting factor. My inability to share ownership was.

Applying Tuckman's model to the full arc: we Formed briefly in wk12, Stormed through wk17–20, reached a fragile Norming by wk21 when the handoff contracts held and the interfaces worked, and arguably never sustained true Performing because repeated external disruptions (weather, hardware failures) threw us back into Storming. We are now in Tuckman and Jensen's (1977) Transforming stage: the project is ending, and I feel the "sense of loss" they describe — not because the project was

a success, but because I can now see the team we could have been if I had invested in trust-building from wk12 instead of trust-avoidance through code.

Graham's cultural competence lecture offers one more frame. On Bennett's (1986) DMIS, I started this project at Minimisation: "we're all engineers, the technical work is what matters." I have moved, imperfectly, toward Acceptance: recognising that my Ukrainian reflex of individual ownership, Robin's preference for simpler interfaces, Edward's insistence on hardware verification, and Demetro's need for spoken-register collaboration are all valid approaches that the team needs to include, not assimilate. The way I'll know I've reached Adaptation — the next stage — is whether I can adjust my working style to a teammate's preferences *before* a conflict forces me to. On this project, every adjustment I made was reactive. The aspiration for the next one is to be proactive.

Falsifiable internship signal: in my first six weeks, does at least one teammate modify a module I own without asking permission? If yes — even once — I have started to become an engineer whose work enables other people's work. If no, I am still the person who writes 2508-line files that only she can read, and I will need to go back to the mechanisms and ask why.

I leave this project believing that the thing I need to develop most is not a skill but a tolerance: the tolerance to watch someone else edit my code, make it worse by my standards, and call that progress. Because a module that two people can maintain at 80% quality is more valuable than a module that one person maintains at 100%. That is an engineering claim, not a feelings claim, and I believe it is true. The evidence: the one file Robin could edit (`passive_watch_2.py`, via the HTTP contract) was the one file that worked on flight day without my intervention. Everything I protected was everything that needed me. That is not a legacy I want to repeat.

What remains unresolved: I do not yet know how to build trust quickly in a team of strangers. The mechanisms I have proposed (pair-programming budget, named-beneficiary test, documentation walkthrough) are process scaffolds, not relationship tools. The relationship part — the part Lencioni says is the foundation of everything — I still find uncomfortable, and I suspect I will continue to find it uncomfortable for years. But I know, now, that the discomfort is data, not a deficiency, and that ignoring it is more expensive than sitting with it. That is the lesson I will carry.

References

Bennett, M.J. (1986). A Developmental Approach to Training for Intercultural Sensitivity. *International Journal of Intercultural Relations*, 10(2), 179–196.

Graham, M. (2025). The Five Dysfunctions of a Team [Lecture video]. AENGM0074 Professional Practice, University of Bristol.

Graham, M. (2025). Innovation Methods: Down-Selection Tools [Lecture]. AENGM0074 Professional Practice, University of Bristol.

Graham, M. (2026). Building Cultural Competence [Lecture video]. AENGM0074 Professional Practice, University of Bristol.

Graham, M. (2026). Assessing and Managing Risk [Lecture video]. AENGM0074 Professional Practice, University of Bristol.

Hatton, N. and Smith, D. (1995). Reflection in teacher education: towards definition and implementation. *Teaching and Teacher Education*, 11(1), 33–49.

Kolb, D.A. (1984). *Experiential Learning: Experience as the Source of Learning and Development*. Englewood Cliffs, NJ: Prentice-Hall.

Lencioni, P.M. (2002). *The Five Dysfunctions of a Team: A Leadership Fable*. San Francisco: Jossey-Bass.

Lencioni, P.M. (2005). *Overcoming the Five Dysfunctions of a Team: A Field Guide*. San Francisco: Jossey-Bass.

Schön, D.A. (1983). *The Reflective Practitioner: How Professionals Think in Action*. New York: Basic Books.

Shore, L.M., Randel, A.E., Chung, B.G., Dean, M.A., Ehrhart, K.H. and Singh, G. (2011). Inclusion and Diversity in Work Groups. *Journal of Management*, 37(4), 1262–1289.

Tuckman, B.W. (1965). Developmental Sequence in Small Groups. *Psychological Bulletin*, 63(6), 384–399.

Tuckman, B.W. and Jensen, M.A.C. (1977). Stages of Small-Group Development Revisited. *Group & Organization Studies*, 2(4), 419–427.